

RAPPORT TECHNIQUE

Projet de Fin d'Année — PFA2

FreelanceHub
Application Mobile Hybride de Freelancing

Section	2GInfo
Date de remise	06 mai 2026
Méthode	Agile Scrum — 6 sprints
Réalisé par	Abbesi Asma, Nasri Safa

Table des matières

Introduction générale	1
1 Cadre général	2
1.1 Contexte et motivation	2
1.2 Objectifs du projet	2
1.3 Méthodologie — Agile Scrum	2
2 Conception	3
2.1 Architecture générale	3
2.2 Modèle de données	3
2.3 Conception des modules principaux	3
2.3.1 Module Authentification	3
2.3.2 Module Profil Freelancer	3
2.3.3 Module Offres et Candidatures	4
2.3.4 Module Paiements et Escrow	4
2.3.5 Module Messagerie	4
2.3.6 Module Administration	4
3 Réalisation	6
3.1 Environnement technique	6
3.2 Organisation du dépôt	6
3.3 Analyse par sprint	6
3.3.1 Sprint 1 — Analyse, rôles, navigation, modèle de données	6
3.3.2 Sprint 2 — Maquettes Figma, authentification, profil freelancer	7
3.3.3 Sprint 3 — CRUD Gigs, messagerie simple, MVP	7
3.3.4 Sprint 4 — Wireframes Offres/Proposals et Store	7
3.3.5 Sprint 5 — Intégration Store, achat simulé, téléchargement	7
3.3.6 Sprint 6 — Admin avancé, optimisation UX, APK final	8
3.4 Bilan de couverture fonctionnelle	8
Conclusion générale	9
Bibliographie	10

Introduction générale

Le marché du travail freelance connaît une croissance exponentielle, portée par la demande croissante de compétences numériques à la demande. Des plateformes telles que Fiverr ou Upwork ont démontré la pertinence d'un modèle mettant en relation directe clients et prestataires indépendants.

FreelanceHub s'inscrit dans cette dynamique en proposant une application mobile hybride développée avec Ionic (Angular) et un backend Flask + MongoDB. Le projet enrichit le modèle standard d'une fonctionnalité innovante : un Store de produits numériques (starter kits, plans d'architecture, modèles IA, code source).

Ce rapport présente l'architecture technique retenue, les décisions de conception, et une analyse honnête de la couverture fonctionnelle réalisée sprint par sprint. Il a pour objectif d'offrir une lecture défendable en soutenance, basée sur le code effectivement livré.

Le document s'organise en trois chapitres : le cadre général du projet, la conception technique, et la réalisation avec analyse par sprint.

Chapitre 1: Cadre général

Ce chapitre présente le contexte du projet, les objectifs poursuivis et la méthodologie adoptée.

1.1. Contexte et motivation

Les plateformes de freelancing génèrent des milliards de transactions annuelles, mais restent peu accessibles sur mobile avec une expérience native. FreelanceHub vise à combler ce manque en proposant une expérience mobile hybride complète, enrichie d'un marché de produits numériques que les plateformes existantes n'intègrent pas nativement.

1.2. Objectifs du projet

- Développer une application mobile hybride (iOS / Android) avec Ionic et Angular.
- Implémenter un backend RESTful avec Flask et MongoDB.
- Couvrir les rôles Freelancer, Client et Admin avec des parcours distincts.
- Intégrer un Store de produits numériques avec achat simulé et téléchargement via Capacitor.
- Produire les livrables Figma (wireframes + maquettes) et un APK fonctionnel.

1.3. Méthodologie — Agile Scrum

Le projet a suivi la méthode Agile Scrum sur une durée de trois mois, découpée en six sprints de deux semaines. Chaque sprint a donné lieu à un incrément livrable accompagné d'une revue de sprint.

TABLE 1.1 – Planning des sprints

Sprint	Durée	Objectif principal
1	S1 – S2	Analyse, rôles, navigation, modèle de données
2	S3 – S4	Maquettes Figma + Authentification + Profil Freelancer
3	S5 – S6	CRUD Gigs + Messagerie + MVP
4	S7 – S8	Wireframes Offres/Proposals + Store
5	S9 – S10	Intégration Store + Capacitor
6	S11 – S12	Admin avancé + UX + APK final

Chapitre 2: Conception

Ce chapitre décrit les choix d'architecture, la modélisation des données et la conception des modules principaux.

2.1. Architecture générale

FreelanceHub repose sur une architecture client/serveur à deux niveaux :

- **Frontend mobile hybride** : Ionic 8 + Angular 20 avec composants standalone.
- **Backend API REST** : Flask 3 + Flask-PyMongo + Flask-JWT-Extended.
- **Base de données** : MongoDB (modélisation par collections documentaires).

La communication entre le front et le back s'effectue exclusivement via des appels REST (`fetch`) avec authentification JWT. Les tokens sont persistés dans le `localStorage` du navigateur mobile. Un mécanisme de fallback `X-User-Id` est implémenté côté backend pour résoudre l'identité en cas d'expiration.

2.2. Modèle de données

La base de données MongoDB comporte les collections suivantes :

2.3. Conception des modules principaux

2.3.1. Module Authentication

- Inscription avec choix du rôle (`freelancer` / `client` / `admin`).
- Hachage des mots de passe avec `bcrypt`.
- Émission de `access_token` et `refresh_token` JWT.
- Guard Angular (`auth.guard.ts`) pour protéger les routes privées.

2.3.2. Module Profil Freelancer

- Lecture et mise à jour via `GET / PUT /api/users/me`.
- Champs couverts : titre, bio, compétences, taux horaire, langues, éducation, expérience, portfolio.
- Synchronisation automatique du portfolio avec les projets terminés.

TABLE 2.1 – Collections MongoDB

Collection	Rôle principal
users	Comptes utilisateurs, rôle, statut actif/bloqué
freelancers	Profil étendu : bio, compétences, portfolio, évaluations
projects	Offres publiées par les clients, avec statut et contrat
applications	Candidatures freelancers avec montant et cover letter
conversations	Fils de messagerie entre client et freelancer
messages	Messages individuels au sein d'une conversation
notifications	Notifications automatiques système
transactions	Escrow, paiements, commissions
payout_requests	Demandes de retrait des freelancers
disputes	Litiges entre client et freelancer
categories	Catégories et tags des offres

2.3.3. Module Offres et Candidatures

- CRUD complet sur les projets/offres publiés par les clients.
- Flux : postulation → acceptation/rejet → contrat → avancement → clôture.
- Filtre PII sur les cover letters (règles regex + fallback LLM optionnel).

2.3.4. Module Paiements et Escrow

- Logique escrow : mise en séquestre → libération → remboursement admin.
- Commission avec essai gratuit sur le premier projet.
- Portefeuille freelancer, demandes de retrait, gestion des litiges.
- Génération de factures PDF sans service externe.

2.3.5. Module Messagerie

- Conversations par fil entre client et freelancer.
- Marquage lu/non lu et notifications à la réception.
- Filtre PII sur les messages.

2.3.6. Module Administration

- Dashboard KPI (utilisateurs, projets, transactions).
- Blocage/activation des comptes utilisateurs.

— Supervision des escrows, retraits et litiges.

Chapitre 3: Réalisation

Ce chapitre décrit l’environnement de développement, l’organisation du code et l’analyse sprint par sprint de la réalisation effective.

3.1. Environnement technique

TABLE 3.1 – Stack technique

Composant	Technologie
Framework mobile	Ionic 8 + Angular 20 (composants standalone)
Backend API	Flask 3 + Flask-PyMongo + Flask-JWT-Extended
Base de données	MongoDB
Authentification	JWT (bcrypt pour le hachage)
Communication API	<code>fetch</code> natif + DTO centralisés (<code>api.dto.ts</code>)
Plugin mobile	Capacitor (prévu pour le Store)
Génération PDF	Bibliothèque Python sans service externe

3.2. Organisation du dépôt

Le dépôt est structuré en deux répertoires principaux :

- `freelancehub-ionic/` : application mobile Angular/Ionic (frontend).
- `freelancehub-backend/` : API Flask (backend).

Côté frontend, les modules Angular couvrent : `auth`, `home`, `client-request`, `service-market`, `project-detail`, `my-jobs`, `messages`, `admin-dashboard`, `payments` et `shared` (DTO, navigation). Côté backend, les blueprints Flask organisent les routes en : `users`, `projects`, `messages`, `notifications`, `payments` et `admin`.

3.3. Analyse par sprint

3.3.1. Sprint 1 — Analyse, rôles, navigation, modèle de données

Les rôles (client, freelancer, admin) sont formalisés dans le middleware `auth.py`. La navigation complète est définie dans `app.routes.ts` et les contrats de données front sont centrali-

sés dans `api.dto.ts`. La structure front/back est cohérente dès ce sprint.

Écart : aucun fichier Figma ou wireframe n'est versionné dans le dépôt.

3.3.2. Sprint 2 — Maquettes Figma, authentification, profil freelancer

L'authentification est fonctionnelle : inscription avec choix du rôle, connexion, hachage `bcrypt`, émission JWT, persistance `localStorage`. Le profil freelancer est complet (bio, compétences, langues, taux horaire, éducation, expérience, portfolio).

Écarts : l'upload de CV PDF n'est pas implémenté. Le workflow *draft* → *pending* → *approved/rejected* pour la validation du profil n'apparaît pas dans le dépôt.

3.3.3. Sprint 3 — CRUD Gigs, messagerie simple, MVP

Le dépôt ne livre pas de module Gigs au sens strict. Il implémente un flux offres/projets complet : publication par le client, candidatures freelancers, acceptation/rejet, passage automatique en *in-progress*. La messagerie (threads, marquage lu/non lu, notifications) est fonctionnelle. Le MVP peut être formulé ainsi : connexion → publication d'offre → candidature → acceptation → messagerie.

Écarts : pas de CRUD dédié aux services/gigs créés par les freelancers. Pas de validation admin avant publication.

3.3.4. Sprint 4 — Wireframes Offres/Proposals et Store

L'expérience utilisateur pour les offres est bien couverte : `client-request`, `service-market` (filtres par catégorie), `project-detail` (soumission de proposition), `my-jobs` (suivi par rôle), `freelancer-profile` (consultation + avis). L'application va au-delà du wireframe en proposant des écrans opérationnels.

Écart : aucun module Store n'apparaît dans le dépôt à ce stade.

3.3.5. Sprint 5 — Intégration Store, achat simulé, téléchargement

Le dépôt ne livre pas le Store demandé. En contrepartie, une brique paiement avancée a été développée : escrow complet, commission avec essai gratuit, libération du paiement, remboursements admin, litiges, portefeuille freelancer, retraits et génération de factures PDF sans dépendance externe.

Écarts majeurs : aucune collection `products` ni page Store. Aucune intégration `Capacitor FileSystem`.

3.3.6. Sprint 6 — Admin avancé, optimisation UX, APK final

L'administration couvre : dashboard KPI, liste et modération des utilisateurs, supervision des transactions, retraits et escrows. L'UX bénéficie d'un design system global (`global.scss`), de dashboards role-based et d'une navigation basse simplifiée.

Écarts : pas de modération de produits (Store absent). Pas de validation admin dédiée aux gigs. Aucun APK final versionné dans le dépôt.

3.4. Bilan de couverture fonctionnelle

TABLE 3.2 – Bilan de couverture fonctionnelle

Fonctionnalité	Statut
Authentification et rôles	Conforme
Profil freelancer	Conforme
Offres et propositions	Conforme
Messagerie	Conforme
Administration (utilisateurs, stats)	Conforme
UX mobile	Conforme
Statuts utilisateur (actif / en attente / bloqué)	Partiel
Validation admin des profils / gigs	Partiel
Figma (wireframes / maquettes)	Non versionné
Upload CV PDF	Absent
Module Gigs (services freelancers)	Absent
Store de produits numériques	Absent
Achat simulé + Capacitor Filesystem	Absent
APK final versionné	Absent

Conclusion générale

FreelanceHub est un projet techniquement abouti sur le flux principal offres-candidatures-messagerie-contrat-escrow-administration. L'architecture Ionic + Flask + MongoDB est propre et maintenable, le parcours client/freelancer fonctionne de bout en bout, et la brique paiement dépasse les exigences initiales.

En revanche, le projet diverge du cahier des charges sur deux blocs majeurs : les Gigs validés par admin et le Store de produits numériques avec Capacitor. Ces modules représentent la valeur différenciante de l'énoncé et leur absence constitue un écart significatif.

Pour un alignement complet, les priorités seraient : (1) créer la collection `products` et les pages Store avec intégration Capacitor Filesystem ; (2) implémenter un module Gigs distinct avec workflow de validation admin ; (3) ajouter l'upload CV PDF et archiver l'APK final.

Cette expérience confirme l'importance d'un suivi sprint par sprint rigoureux pour éviter que des choix techniques pertinents ne dérivent des livrables attendus. Les compétences acquises sur l'architecture mobile hybride, la gestion des rôles et la logique escrow constituent une base solide pour les développements futurs.

Bibliographie

1. Ionic Team. *Ionic Framework Documentation*, 2024. [En ligne]. Disponible : <https://ionicframework.com/docs>. [Consulté le 06 mai 2026].
2. Pallets Projects. *Flask Documentation*, v3.0, 2024. [En ligne]. Disponible : <https://flask.palletsprojects.com>. [Consulté le 06 mai 2026].
3. MongoDB Inc. *MongoDB Manual*, 2024. [En ligne]. Disponible : <https://www.mongodb.com/docs>. [Consulté le 06 mai 2026].
4. Ionic Team. *Capacitor Documentation*, 2024. [En ligne]. Disponible : <https://capacitorjs.com/docs>. [Consulté le 06 mai 2026].
5. Google LLC. *Angular Documentation*, v20, 2024. [En ligne]. Disponible : <https://angular.io/docs>. [Consulté le 06 mai 2026].